MEDIUM TRACK — TOURNAMENTLAB

Build the ultimate Esports League Manager | Python engineering challenge

FOR: learners who can explain variables, conditions, loops, functions, lists and dictionaries and can build small CLI programs, but are still learning architecture, rigorous tests, algorithms and collaboration. This is a new competitive-game project, not a calculator or CampusFlow. Suggested learning/build time: 12–18 focused hours.

YOUR MISSION — BECOME THE LEAGUE ENGINEER

A gaming club wants a fair competition: teams register, a round-robin fixture is generated, scores are entered, and an automatically updated league table decides the champion. Build TournamentLab, a robust Python CLI that handles real tournament rules, rejected results, rankings, persistence and an end-of-season champion report.
You are responsible for making the league FAIR. A score entry must not corrupt the table; duplicated fixtures must not award points twice; tied teams must be sorted predictably. No website or esports API is required: everything uses locally supplied team names and match results.

Required features

1. Register 4–8 unique teams with trimmed display names (case-insensitive uniqueness). 2. Create a single round-robin schedule (every pair meets once; each team plays N−1 games). 3. List fixtures by ID, team names and status pending/played. 4. Record nonnegative integer scores for a scheduled fixture once. 5. Derive standings from match records: played, wins, draws, losses, goals for, goals against, goal difference, points. 6. Award 3 points for a win, 1 each for a draw and 0 for a loss. 7. Rank by points descending, goal difference descending, goals for descending, then normalized team name ascending. 8. Save/load JSON and reject inconsistent data. 9. Support 'simulate demo fixtures' only with an explicit choice and seeded reproducible inputs; do not secretly change real records. 10. Print a champion once all fixtures have results; otherwise label the standings 'in progress'.

Engineering constraints

Python standard library, CLI and unittest. Use modules/functions rather than a massive main.py. No external packages necessary. No AI API; AI is used as a research tutor and reviewer.

Prohibited shortcuts: hardcoded league standings, awarding points during display without consistency checks, duplicate result submission, changing a completed score without an explicit documented edit operation (not required), storing unvalidated arbitrary Python objects.

STAGE 0 — BEGIN WITH A DIAGNOSTIC

On paper, for three teams Red, Blue, Green, list all unique pairs. Now compute a simple table after Red 2–0 Blue, Blue 1–1 Green, Red 0–1 Green. Do not use AI or code initially. If you cannot explain where wins and points come from, study Lessons 1–3 before writing implementation.

STAGE 1 — WHAT TO REVISIT, WHAT TO LEARN

Lesson 1 | Core Python data types, nested structures and safe access

REVISIT: strings, int, bool, list, dict, for loop, membership, list/dict iteration, .get(), enumerate. LEARN: model related data instead of parallel lists; immutability vs mutation; why names are display labels but IDs are stable identifiers. MINI-LAB: represent three movies as dictionaries, search by ID, reject duplicate IDs. CHECK: why should renaming a team not change match references?

Lesson 2 | Functions and clean architecture

REVISIT: def, return, parameters, scope, comprehensions. LEARN: pure functions vs side effects, explicit contracts, minimizing mutable globals, modular design. MINI-LAB: write calculate_average(scores) and test empty values. CHECK: Why should compute_standings(teams,matches) return data rather than print and save a file?

Lesson 3 | Nested loops and combinatorics

REVISIT: range, for, nested loops and if. LEARN: unique unordered pairs, N*(N−1)/2, avoiding self-matches and reversed duplicates. MINI-LAB: from names ['A','B','C','D'] produce each two-name pair exactly once without itertools, then compare with itertools.combinations. CHECK: how many fixtures for N=6? Answer only after deriving the formula.

Lesson 4 | Sorting, tie-break rules and functions as arguments

REVISIT: sorted(), key=, reverse, numeric and alphabetical comparison. LEARN: lexicographic multi-key sorting, stable ordering and sign inversion for descending numeric values. MINI-LAB: sort sample books by rating descending then title alphabetically. CHECK: How can two teams with equal points have different ranks?

Lesson 5 | Input validation and exceptions

REVISIT: .strip(), .casefold(), int conversion, ValueError, try/except. LEARN: strict integer checks (bool is not allowed), reject a score <0, distinguish unknown fixture, already played fixture and valid fixture, validate-before-mutate, no partial results. MINI-LAB: validate two nonnegative integers from unrelated sports scores. CHECK: does `isinstance(True,int)` pass; how will you avoid treating True as a goal?

Lesson 6 | Aggregation and derived state

REVISIT: accumulator variables, dictionaries, nested loops. LEARN: compute a league table from a canonical list of results; reconstruct standings from source of truth to avoid drift. MINI-LAB: compute per-person totals from grocery purchases by customer; compare with hand arithmetic. CHECK: why is storing both matches and independently editable standings risky?

Lesson 7 | JSON persistence, schema and ID continuity

REVISIT: json.dump, json.load, pathlib, file modes. LEARN: versioned structure, schema validation, uniqueness, referential integrity (fixture team IDs must exist), parse into temporary data before replacing active state. MINI-LAB: save/restore a small book catalog and refuse duplicate IDs. CHECK: what would happen if a saved fixture mentions a team that does not exist?

Lesson 8 | Automated tests and edge-case design

REVISIT: unittest.TestCase, setUp, assertEqual, assertRaises; learn temporary files via tempfile.TemporaryDirectory, test fixtures, table-driven cases, test coverage reasoning (not just test count). MINI-LAB: test tie-break sorting with at least four crafted records. CHECK: can all demo games pass while a reversed fixture causes duplicate points?

Lesson 9 | Debugging and professional Git

LEARN: debug a failing assertion, isolate a minimal reproducible example, git branch workflows, GitHub Issues, PR review, review comments, conflicts and merge tests. MINI-LAB: two fellows propose two implementations of fixture IDs, compare trade-offs and document one decision. CHECK: what evidence shows a review happened BEFORE merge?

STAGE 2 — REQUIREMENTS AND DESIGN WORKSHOP

Canonical record shapes (illustrative, not full implementation)

Team: {'id':'TM001','name':'Red Comets'}. Fixture: {'id':'M001','home_id':'TM001','away_id':'TM002','status':'pending','home_score':None,'away_score':None}. Played fixture uses status='played' and two nonnegative integer scores. Use separate dictionaries/lists or classes only if you understand them; basic functions and dicts are sufficient.

Agreed rules

Team name: 2–30 characters after trim; casefolded names unique. 4–8 teams before generation; fixtures generated only once per season. Each unordered pair of teams occurs once; home vs away order can be arbitrary but deterministic. Match IDs M001… unique. Only pending fixture can accept a score. No score entry is allowed before a fixture exists. League standings derive from PLAYED matches only. A league is complete only if every fixture has been played.

Data invariants

Every fixture references two distinct existing teams. Every fixture ID is unique. For pending fixtures both scores are None; for played fixtures both scores are integers >=0 and neither is boolean. No duplicate unordered team pair. Total league points over all played matches = 3*(non-draw matches) + 2*(draw matches); total goals for = total goals against. A rejected score must not change any record.

Ranking example

If Team X and Team Y both have 7 points, compare goal difference, then goals scored, then normalized team names alphabetically. This tie-break order must be documented and tested; do not silently introduce head-to-head rules.

STAGE 3 — BUILD ROADMAP

Mission A | Register and validate teams

Build register_team, list_teams and duplicate-name checks. Commit and write tests before moving forward. DEMO: Red Comets registers; ' red comets ' is rejected as duplicate. STOP: why should display name and identifier be separate?

Mission B | Generate a fair fixture list

Generate every unique unordered team pair exactly once. For six teams expect 15 fixtures; for four expect six. Fixture IDs are deterministic. DEMO: no self-fixtures, no reverse duplicates, same count each rerun under identical input. STOP: explain N*(N−1)/2 with a drawing.

Mission C | Record results safely

Ask for fixture ID and two scores. Verify fixture exists and pending, then update BOTH scores and status in one successful operation. DEMO: M001=2:1 accepted, second submission rejected, invalid score rejected with no mutation. STOP: what is an atomic update in this simple in-memory setting?

Mission D | Compute standings on demand

Compute standings from teams plus played fixtures. Do NOT store a separate mutable scoreboard that can contradict the matches. DEMO: win/draw/loss points, goals for/against, goal diff and ranking tie rules. STOP: what is the source of truth?

Mission E | Save/load and produce league report

Save teams, fixtures and schema_version=1. On load validate required keys, team IDs, names, status/score consistency, unique pairs and unique IDs before replacing memory. DEMO: saved season restores exact standings; malformed fixture reference rejected. STOP: why do valid JSON syntax and valid business data mean different things?

Mission F | Friendly terminal experience and README

Add menu: register team; list teams; generate schedule; list fixtures; record result; standings; save; load; show champion; quit. Build clear success and error messages, repeat invalid input, never crash from empty/invalid choices. README includes commands, tournament rules, tie-breakers, tests and example season.

STAGE 4 — PROFESSIONAL TEST LAB

At least 14 meaningful unittest tests. Required scenarios: 1 reject duplicate casefolded team name; 2 reject blank/too-long name; 3 reject fixture generation with fewer than 4 teams; 4 four teams produce six fixtures; 5 six teams produce fifteen fixtures; 6 no self-match; 7 no duplicate unordered pair; 8 reject negative/float/bool goals; 9 reject unknown fixture; 10 reject duplicate result; 11 correct 3/1/0 points; 12 tie-break sorting; 13 standings re-computed from matches; 14 JSON bad references rejected without state change; 15 reload preserves IDs and standings; 16 champion shown only when all matches played. The first 14 are minimum; completing 16 is encouraged.

Controlled demonstration dataset

Create four teams: Thunder, Comets, Phoenix, Rockets. Six fixtures total. Enter scores for exactly two fixtures, one win and one draw, and verify displayed table manually. Submit an intentionally invalid score and confirm no standings change. Reload from JSON, verify the same table, then complete remaining fixtures and display winner.

Cross-check questions

If a game ends 0–0, should both teams receive one point? If a team loses 3–2, does it still get two goals for? Can two teams occupy the same points and GD, and how do you break the tie? Can a corrupt JSON file create a fixture with a phantom team? Why should rendering standings not modify fixtures?

STAGE 5 — ENGINEER A / ENGINEER B WORK SPLIT

Engineer A — Scheduling and reliable data layer

Own team registration/validation, unique IDs, round-robin generation, persisted format and load validation. Write at least five tests covering registrations, fixture count, duplicate fixtures, invalid file. Create branch feat/1-team-scheduling and one PR; review B's PR with meaningful comments. DEFENCE: derive fixture formula, show duplicate rejection, prove file-load consistency.

Engineer B — Scoring, standings and CLI

Own result entry/validation, standings calculation, sorting and tie-breaks, champion report and CLI. Write at least five tests covering score validation, points, sorting, champion. Create branch feat/2-results-and-rankings and one PR; review A's PR meaningfully. DEFENCE: show table recomputation and explain sorting keys.
Both: decide contract function names, error handling and data shapes before work. Each authors >=2 substantive commits, code plus tests, and reviews/approves the OTHER's PR before merge. No self-approval or unreviewed main pushes. Run all tests again on integrated main. Solo learners can ask mentor for PR review.

STAGE 6 — MASTER AI PROMPTS FOR DEEP LEARNING

MASTER PROMPT M1 — CONCEPT BOOTCAMP

Act as a senior Python engineer training an intermediate beginner. I am designing TournamentLab, but I will write all project logic. Teach me [TOPIC: nested loops / combinations / pure functions / sorting / dictionaries / exceptions / JSON validation / unittest] using an UNRELATED small example. Divide into five stages: explain; ask me to predict output; give a tiny independent drill; review my answer; then test me with a tricky edge case. Do not show project-ready implementation unless I first submit an attempt. Ask one question at a time, wait for my answer, and refuse to move on until I can explain my reasoning.

MASTER PROMPT M2 — TECHNICAL EXAMINER THAT CHECKMATES ME

Act as a rigorous software engineering interviewer. Test whether I truly understand [TOPIC] and can use it for TournamentLab. Ask 12 escalating questions ONE AT A TIME: tracing loops and dictionaries, hand-calculating a table, sorting tie-breaks, invalid-input cases, test design and explaining complexity. Each answer receives a 0–3 mastery rating with reasons. Ask a deeper follow-up when I seem to guess. Do not flatter or reveal answers early. At the end give an evidence-based mastery table, missing concepts, two next drills and one small live-coding challenge.

MASTER PROMPT M3 — REQUIREMENT REVIEW BEFORE I CODE

I am writing TournamentLab. Here is my own proposed data model and functions: [PASTE DESIGN]. Here are the exact business rules: 4–8 teams; each pair once; 3/1/0 points; rank by points, GD, GF, normalized name; JSON reload must reject invalid records. Do not write source code. Identify ambiguous assumptions, edge cases, state invariants, unnecessary mutable state, and a minimal set of verification tests. Ask me five design questions I must answer. Let me make the design decisions myself.

MASTER PROMPT M4 — DEBUGGER AND TEST COACH

I have a failing TournamentLab test. Requirement: [RULE]. Minimal code: [CODE]. Actual traceback/output: [OUTPUT]. Expected output: [EXPECTED]. First ask me for a hypothesis, then propose exactly three experiments and explain what each result would prove. Help me isolate the problem without replacing my code. After I attempt a fix, check the test, ask for a regression test and explain whether any invariant could still fail.

MASTER PROMPT M5 — EVIDENCE-BASED REPOSITORY AUDIT

You are an independent Python reviewer. I will provide TournamentLab files in multiple messages, ending with END PROJECT; do not review until then. Audit: requirements, schedule uniqueness, fixture count N*(N−1)/2, team/fixture IDs, score validation, ranking tie breaks, persistence schema, exception handling, 14+ meaningful unittest tests, CLI usability, README, Git branch/PR evidence. Never claim to have run tests or inspected GitHub links unless you truly have access/results; use NOT VERIFIED otherwise. Return a compliance matrix with PASS / FAIL / NOT VERIFIED; issues P0–P3; exact function and reproduction steps; missing tests; small guidance instead of full replacement code; and 8 questions that would expose whether the author understands each module. Ask for missing evidence.

MASTER PROMPT M6 — AI OUTPUT SKEPTICISM

Here is an AI suggestion for my fixture generation or league ranking: [PASTE SUGGESTION]. Challenge it. Construct three counterexamples and ask me to predict what the suggestion will do. Separate facts from assumptions; propose tests I can actually run. Do not declare it correct until experimental evidence is provided. Then compare a simpler alternative without handing me a finished solution.

STAGE 7 — DOCUMENTATION AND PROFESSIONAL HABITS

Repo structure suggestion: main.py; tournament/{teams.py,fixtures.py,standings.py,storage.py,cli.py}; tests/; docs/{design-decisions.md,ai-learning-log.md}; README.md; .gitignore. Any clean equivalent accepted. Tests should be deterministic and use tempfile for saved files. Do not commit generated test data containing private information. Use meaningful GitHub Issues and focused commits.

Your learning log must prove the learning happened

Each engineer records 3 AI learning cycles: Topic I did not know → exact prompt → AI's explanation in my own words → tiny independent experiment → actual results → corrected misconception/decision → related code/test/commit. At least one cycle per engineer includes choosing not to use an AI suggestion or changing it after verification, if such an occasion genuinely occurs; otherwise document the evidence-backed alternative rather than inventing a hallucination.

PR review questions

Does every unordered team pair appear once? Is the tie-break sort correct? Is bool incorrectly accepted as an integer score? Does the load validator accept a fixture referencing an unknown team? Can I reproduce the same standings from the same saved matches? What regression test protects a recent fix?

STAGE 8 — FINAL PROFESSIONAL DEFENCE

Individually explain: how fixture generation avoids duplicates; why standings should be computed from results; how you proved multi-key sort order; what happens on duplicate score entry; how load validation prevents memory corruption; the most important review comment you received and the AI recommendation you independently checked. Demonstrate two success cases, two invalid cases, a save/reload round trip, test suite and both PR timelines.

Scoring rubric — 100 marks

Team registration + fixtures 20; correct scoring and standings 25; data integrity and JSON 15; CLI and user experience 10; tests and debugging 15; Git/PR professionalism 5; independent AI learning, README and individual technical defence 10. Functional completion alone does not earn a professional grade.

Submission

Repo link, main commit SHA, Engineer A and B names, two reviewed PR links, demo data and screenshot/text of standings, automated test command + output, README, docs/design-decisions.md, docs/ai-learning-log.md, and two personal reflections about the toughest concept learned and one bug discovered by testing.